Smart City Transportation Network Optimization

Code-Focused Technical Report

Stack: Python, Streamlit, Plotly Mapbox, pandas, scikit-learn, Docker

1. Technical Overview

This project is a Python smart-city transportation system that combines algorithms with an interactive Streamlit dashboard. Cairo is modeled as a weighted graph where nodes are locations and edges are roads. The system applies graph algorithms, dynamic programming, greedy optimization, and machine learning to transportation planning problems.

The application is integrated rather than a collection of separate scripts. The graph is loaded once, shared across modules, visualized through reusable Plotly map logic, and exposed through a multi-page Streamlit UI.

- UI layer: app.py handles navigation, KPI cards, tabs, maps, and user controls.
- Graph layer: data_loader1.py loads nodes.csv and edges.csv into CairoGraph.
- Algorithm layer: dijkstra.py, astar.py, kruskal_mst.py, dp_maintenance.py, public_transit_scheduler.py, and greedy.py.
- Prediction layer: traffic_ml_model.py trains and serves Random Forest traffic predictions.
- Deployment layer: Dockerfile, requirements.txt, and .dockerignore package the project.

2. Code Architecture

2.1 Graph Model

CairoGraph stores nodes, edges, and adjacency_list. nodes is a dictionary indexed by node ID, edges is a list of Edge objects, and adjacency_list maps each node to outgoing road edges.

```
graph.nodes         # { node_id: Node }
graph.edges         # all Edge objects
graph.adjacency_list # { node_id: [Edge, ...] }
```

The graph loader creates reverse edges for undirected road traversal. Without reverse adjacency entries, Dijkstra or A* could fail to find routes that should exist in the physical road network.

2.2 Streamlit State And Caching

The app uses st.session_state for route results, active page, chaos simulation edges, selected time of day, and UI choices. The graph is loaded with st.cache_resource so it is not rebuilt on every widget interaction.

```
@st.cache_resource(show_spinner=False)
def load_graph():
    graph = CairoGraph()
    graph.load_data(nodes_path, edges_path)
    return graph
```

3. Algorithm Implementation Details

3.1 Dijkstra Routing

Dijkstra finds the shortest or fastest path between two nodes using non-negative edge weights. The weight can come from static distance or from graph.get_weight() when the traffic predictor is active. With a priority queue, complexity is O((V + E) log V).

3.2 A* Emergency Routing

A* uses f(n) = g(n) + h(n), where h(n) is a geographic distance heuristic. This helps emergency routing because the search is guided toward the hospital and usually explores fewer nodes than Dijkstra.

3.3 Kruskal MST Road Expansion

Kruskal's algorithm uses Union-Find with path compression and union by rank. Existing roads are connected first, then potential roads are sorted by construction cost. The dominant cost is sorting, so complexity is O(E log E).

```
if uf.union(edge.from_id, edge.to_id):
    chosen_roads.append(edge)
    total_cost += actual_cost
```

3.4 Maintenance DP Knapsack

Road maintenance is a 0/1 Knapsack problem. Each candidate road has repair_cost and repair_value. The DP table has dimensions (n + 1) by (budget + 1). Backtracking through the table returns the exact roads selected for repair.

```
dp[i][w] = dp[i - 1][w]
if w >= cost:
    dp[i][w] = max(dp[i][w], dp[i - 1][w - cost] + value)
```

3.5 Bus Allocation DP

The bus scheduler uses dynamic programming to distribute a limited fleet across routes. This avoids the weakness of a greedy approach that might assign too many buses to the single highest-demand route.

3.6 Greedy Signal Control

The signal controller uses a greedy decision rule for fast local optimization. It is useful for real-time simulation, but it can be suboptimal if downstream congestion is ignored.

4. Code-Level Problems Faced And Fixes

4.1 Maintenance DP Used A Different Visualization Path

The Maintenance DP algorithm produced correct values, but its visualization did not match the rest of the app. It was displayed separately from the shared Plotly Mapbox renderer, causing inconsistent styling and interaction.

Fix: build_map() in app.py was extended with a maintenance_result parameter so maintenance can reuse the same Cairo basemap, node rendering, layout, and Streamlit map behavior.

```
def build_map(..., maintenance_result=None):
    maintenance_mode = maintenance_result is not None
```

4.2 Selected Repair Roads Were Hard To Identify

After moving maintenance to a real map, selected roads still needed clear emphasis. If all roads were only condition-colored, the DP decision was not obvious.

Fix: selected roads are stored as sorted endpoint tuples and rendered in cyan with thicker lines and a glow layer.

```
selected_maintenance.add(tuple(sorted([road['from_id'], road['to_id']])))
```

4.3 Zoom Was Not Consistent Across Maps

Plotly maps support zoom, but scroll zoom must be explicitly enabled when the toolbar is hidden. Initially, only the maintenance map had it enabled.

Fix: a shared configuration was created and applied to Trip Planner, Emergency Response, MST, and Maintenance maps.

```
PLOTLY_MAP_CONFIG = {'displayModeBar': False, 'scrollZoom': True}
```

4.4 Directed vs Undirected Road Access

edges.csv lists each road once, but most city roads should be traversable both ways. If only forward edges are stored, routing can fail depending on direction.

Fix: the loader stores both forward and reverse adjacency entries. Unique-edge helpers prevent duplicate display on maps.

4.5 Existing Roads Mixed With Potential Roads

The same edge file contains built roads and future candidate roads. Routing must not accidentally use unbuilt roads, while MST must evaluate them.

Fix: Edge exposes is_existing and is_potential based on Cost. Routing defaults to existing roads, while MST uses potential roads intentionally.

```
edge.is_existing  # cost == 0
edge.is_potential # cost > 0
```

4.6 DP Budget Required Integer Indexes

The maintenance DP table indexes budgets from 0 to W. Floating repair costs would break safe indexing.

Fix: repair_cost is converted to int after calculation, making the DP table deterministic and index-safe.

4.7 Streamlit Reruns Could Reset UI Output

Streamlit reruns the script after widget interactions. Without session state, route results and selections could disappear.

Fix: session state defaults are initialized for route_result, active_page, selected algorithm, selected time, potential-road toggle, and chaos edges.

4.8 Docker Slim Image Needed System Libraries

A slim Python Docker image can miss Linux libraries needed by matplotlib, scikit-learn, or graphical dependencies.

Fix: the Dockerfile installs build-essential, libglib2.0-0, and libgl1 before installing Python dependencies.

4.9 Cloud Deployment Uses Dynamic PORT

Cloud platforms often inject a PORT environment variable. Hardcoding only 8501 can break deployment.

Fix: the Docker CMD uses ${PORT:-8501}, so local Docker uses 8501 while cloud platforms can override it.

```
CMD streamlit run app.py --server.port=${PORT:-8501} --server.address=0.0.0.0
```

5. Map Rendering Implementation

build_map() creates a Plotly Figure, draws road edges, overlays routes or maintenance selections, groups nodes by type, and applies the carto-darkmatter basemap. Reusing this function keeps all map pages consistent.

- Normal mode: congestion colors, route glow, route waypoints, and chaos overlays.
- Maintenance mode: condition colors, cyan repair overlay, and repair hover details.
- Shared layout: Cairo center, dark basemap, consistent legend, hover labels, and height.

This is better than maintaining separate map functions because future map improvements can be made once and reused everywhere.

6. Dockerization Details

The Dockerfile uses python:3.11-slim, installs Linux packages, installs requirements.txt, copies project files, writes Streamlit configuration, exposes port 8501, and runs app.py.

```
docker build -t smart-city-transportation .
docker run --rm -p 8501:8501 smart-city-transportation
```

.dockerignore excludes cache folders, virtual environments, editor settings, logs, zip files, and local Streamlit settings. The recent map fixes did not add dependencies, so requirements.txt did not need to change.

7. Testing And Validation

Testing was done at syntax, algorithm, runtime, and UI levels.

```
python -m py_compile app.py dp_maintenance.py
```

- Syntax validation confirmed app.py and dp_maintenance.py compile successfully.
- A runtime smoke test built the maintenance map and confirmed selected road count and score values.
- The app was opened locally at localhost:8501.
- The Infrastructure page and Maintenance DP tab were checked in the browser.
- Maintenance displayed a real Cairo basemap with selected roads highlighted.
- Zoom was enabled on all real map views.

A complete Docker validation would build the image, run the container, then repeat the same browser checks at localhost:8501.

8. Complexity Summary

- Dijkstra: O((V + E) log V) with a priority queue.
- A*: usually explores fewer nodes when the heuristic is informative.
- Kruskal MST: O(E log E), dominated by sorting.
- Maintenance DP: O(nW), where n is roads and W is budget.
- Bus allocation DP: depends on route count and fleet size.
- Greedy signal control: fast local choice, approximately O(k) for k incoming roads.

9. Conclusion

The project successfully combines multiple algorithmic techniques into one smart-city transportation dashboard. The biggest recent improvement was fixing the Maintenance DP visualization and making map interaction consistent across the application.

The challenges were mostly integration issues: connecting algorithm results to shared visualization, preserving Streamlit state, separating existing roads from candidate roads, handling Docker runtime dependencies, and supporting deployment ports. Solving these issues made the system more professional, reliable, and easier to use.